

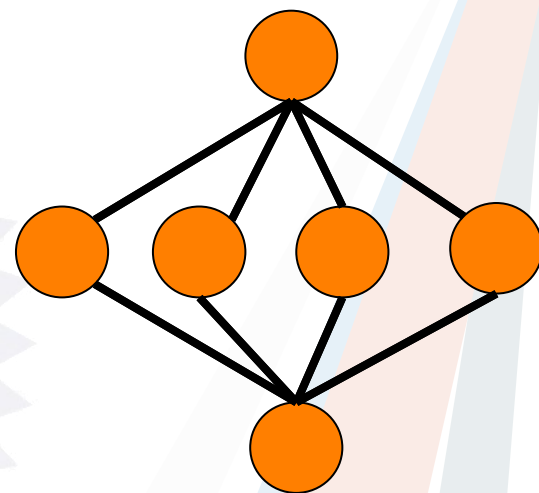
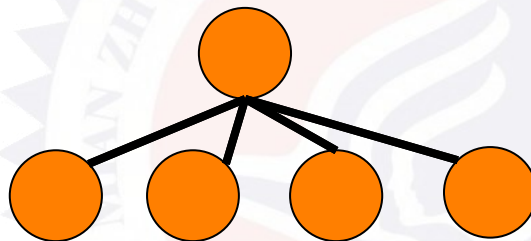
图论基础

主讲：黄凤林



→ 数据结构

维护数据与数据之间的关系的存储结构



线性类数据结构：

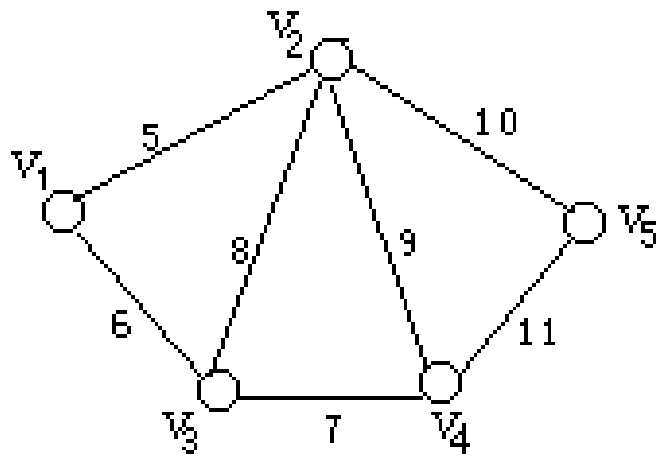
- ✧ 数组：任意位置访问存储
- ✧ 栈：一端操作（查询、插入、删除等）
- ✧ 队列：两端操作（队尾进、队首出）
- ✧ 链表：每次查询从头开始，插入和删除比较方便



图的基本概念

1、图的的定义

图是由顶点 V 的集合和边 E 的集合组成的二元组：
记 $G = (V, E)$





基本概念

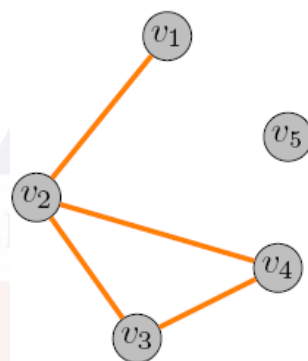
- ✧ 二元组 (V, E) 称为图(graph)。 V 为顶点集， E 为 V 中结点之间的边的集合。
- ✧ 自环：一条边的两个端点是相同的。
- ✧ 多重边：两个端点之间有两条以上的边，称他们是多重边。
- ✧ 简单图：没有自环和多重边的图



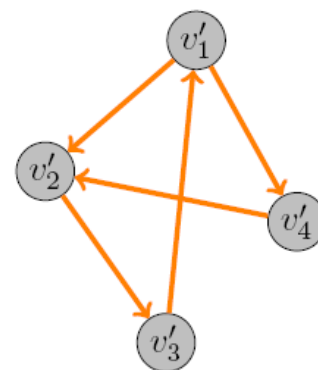
基本概念

- ✧ 无向边：边是双向的
- ✧ 有向边：单向边，有箭头
- ✧ 无向图：只有无向边的图
- ✧ 有向图：只有有向边的图
- ✧ 顶点的度

- 无向图中，一个顶点相连的边数称为该顶点的度。
- 有向图中，从一个顶点出发的边数称为该顶点得出度；到达该顶点的边数称为它的入度。



(a) 无向图

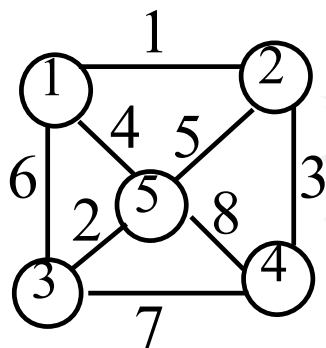


(b) 有向图

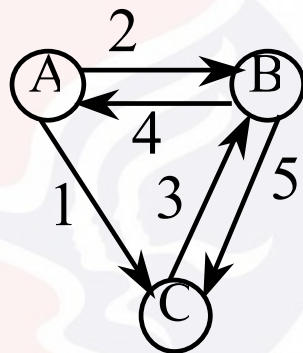
基本概念

➤ 权和网

在图的边给出相关的数，成为权。权可以表示一个顶点到另一个顶点的距离，耗费等，带权图一般称为网。



(a) 无向网



(b) 有向网



基本概念

- ✧ 完全图、稠密图和稀疏图
- ✧ 完全图：
 - ✧ 任何两个顶点之间都有边（弧）相连称为完全图
- ✧ 稀疏图、稠密图：
 - ✧ 边（弧）很少的图称为稀疏图反之为稠密图

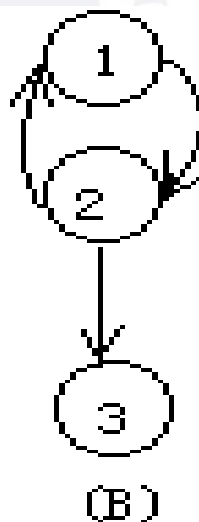
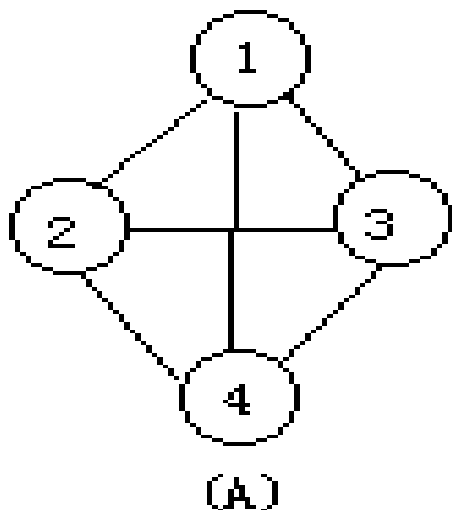
NOI

简单路径和回路

如果一条路径上的结点除起点 x_1 和终点 x_k 可以相同外，其它结点均不相同，则称此**路径为一条简单路径**。

图(a)中 $v_1 \rightarrow v_2 \rightarrow v_3$ 是一条简单路径， $v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_1 \rightarrow v_3$ 不是简单路径。 **$x_1 = x_k$ 的简单路径称为回路（也称为环）**。

例如图(b)中， $v_1 \rightarrow v_2 \rightarrow v_1$ 为一条回路。





图的存储-邻接矩阵

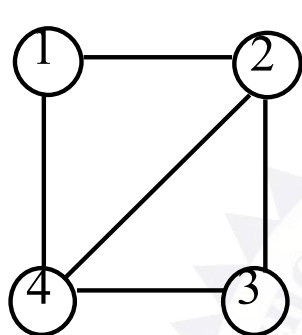
在邻接矩阵表示中，除了存放顶点本身信息外，还用了一个矩阵表示各个顶点之间的关系。若 $(i,j) \in E(G)$ 或 $\langle i,j \rangle \in E(G)$ ，则矩阵中第 i 行 第 j 列元素值为 1，否则为 0。

图的邻接矩阵定义为：

$$A[i][j] = \begin{cases} 1, & \text{若 } (i,j) \in E(G) \text{ 或 } \langle i,j \rangle \in E(G) \\ 0, & \text{其它情形} \end{cases}$$

图的存储—邻接矩阵

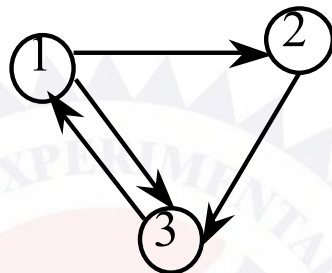
例如, 下面为两个无向图和有向图对应的邻接矩阵。



(a) 无向图 G3

$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

(a) G3 的邻接矩阵



(b) 有向图 G4

$$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

(b) G4 的邻接矩阵

图的存储—邻接矩阵

类似地可以定义网的邻接矩阵为：

$$A[i][j] = \begin{cases} w_{ij} & \text{若 } (i,j) \in E(G) \text{ 或 } \langle i,j \rangle \in E(G) \\ 0 & \text{若 } i=j \\ \infty & \text{其它情形} \end{cases}$$



→ 图的存储—邻接矩阵

空间复杂度： $O(V^2)$

优点：直观，容易理解，可以直接查看任意两点的关系。

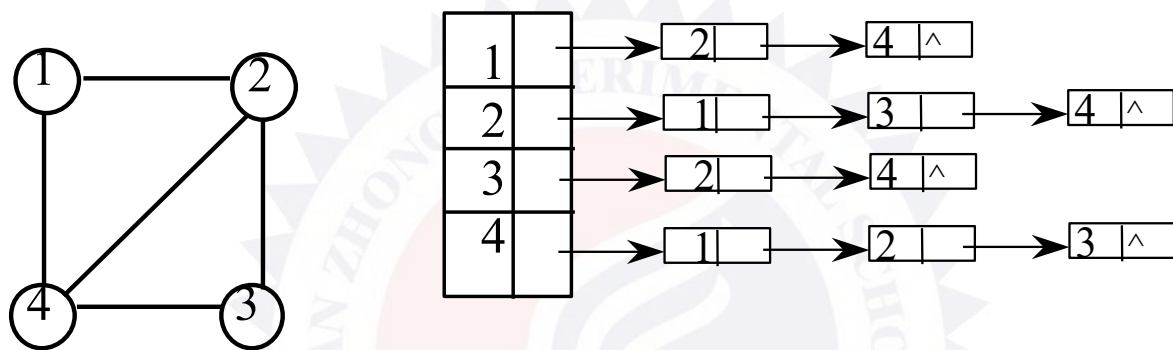
缺点：对于稀疏图，会有很多空间根本没有利用。对于带权图，不能处理重边。要查询某一个顶点的所有边，要枚举 V 次。

NOI



图的存储-邻接表

对图的每个顶点建立一个单链表(n 个顶点建立 n 个单链表), 第 i 个单链表中的结点包含顶点 V_i 的所有相邻接点。



(a) 无向图 G_3 的邻接表

空间复杂度：有向图 $O(V+E)$ 无向图 $O(V+2*E)$

优点：节省空间，能快速找到某个顶点所有相连的顶点，而无需访问无关顶点。



邻接表实现（伪链表）

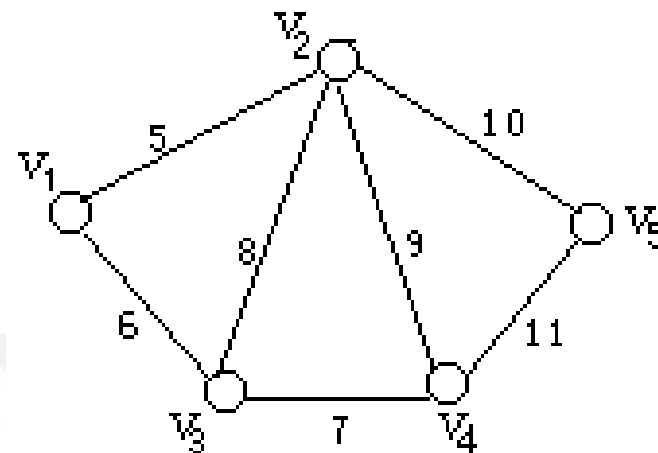
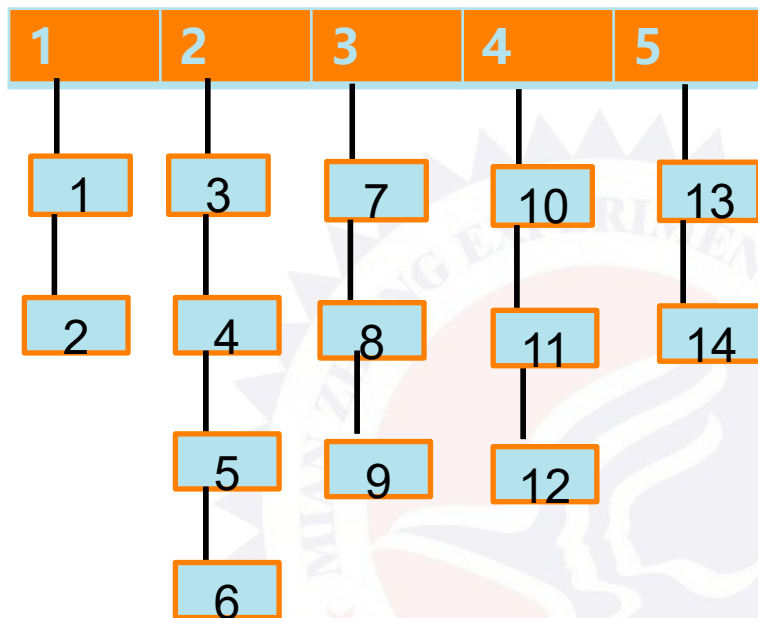
```
struct node{
    int to,next;
};
node edge[100];
int head[20];
int k=0;
void adde(int u,int v){
    edge[++K].to=v;
    //edge[k].w=w;
    edge[K].next=head[u];
    head[u]=K;
}
```

```
void dfs(int u){//深度优先遍历
    printf("%d ",u);
    visit[u]=true;
    for(int i=head[u];i;i=edge[i].next)
    {
        if(!visit[edge[i].to])
        {
            dfs(edge[i].to);
        }
    }
}
```



邻接表实现（伪链表）

Head[]



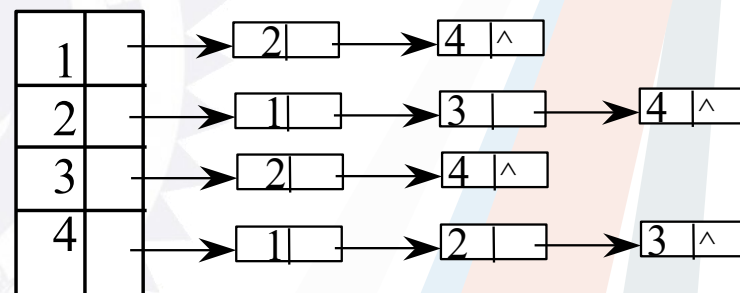
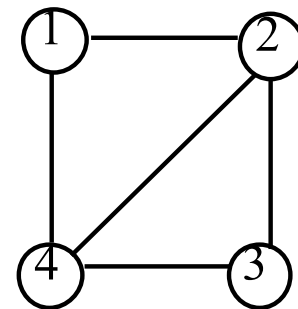
Edge[]

1	2	3	4	5	6	7	8	9	10	11
v2	v3	V1	V3	V4	V5	v1	v2	v4	v3	v2
5	6	5	8	9	10	6	8	9	7	9

邻接表实现 (指针)

```
struct node{
    int data;
    node *next;//相邻地址
};
node first[30000+1];//邻接表表头
node *t; int n,m;
```

```
void init()
{
    scanf("%d%d",&n,&m);
    for(int i=1;i<=m;i++){
        int u,v,w;
        scanf("%d%d%d",&u,&v,&w);
    }
}
```



(a)无向图 G3 的邻接表



→ 邻接表实现 (指针)

```
void insert(int u,int v,int w)
{
    t= new (node);
    t.data=v;
    t.w=w;
    t->next=first[u]->next;
    first[u]->next=t;
}
```

NOI



→ 图的遍历

给出一个图 G 和其中任意一个结点 V_0 ，从 V_0 出发访问 G 中所有结点，每一个结点被访问一次，这就叫图的遍历：

通常有两种遍历方法：

(1) 深度优先搜索dfs

(2) 广度优先搜索bfs

➔ 深度优先遍历(DFS)

遍历算法：

- 1、从某一顶点出发开始访问,被访问的顶点作相应的标记,输出访问顶点号。
- 2、从被访问的顶点出发,搜索与该顶点有边的关联的某个未被访问的邻接点。

再从该邻接点出发进一步搜索与该顶点有边的关联的某个未被访问的邻接点,直到全部接点访问完毕

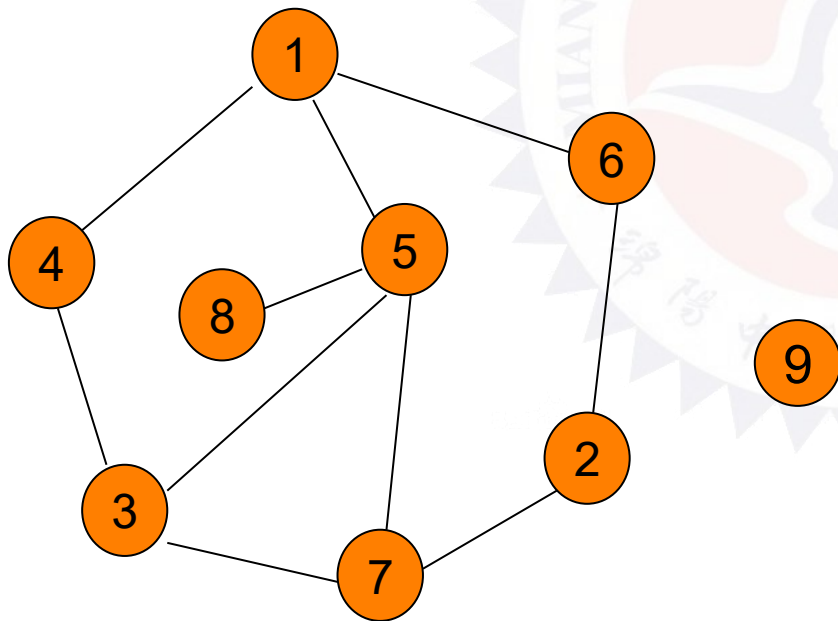


➔ 广度优先搜索(BFS)

广度优先搜索按层次遍历的过程，其搜索过程如下：

假设从图中某结点 v_0 出发，在访问了 v_0 之后依次访问 v_0 的各个未曾访问的邻接点，然后分别从这些邻接点出发按广度优先搜索的顺序遍历图，直至图中所有可被访问的结点都被访问到。

依次访问和 v_0 有路径相连且路径长度为1, 2, 3……的结点



如何，从1号结点开始广度遍历，则遍历顺序为：

1-->4-->5--6-->3-->8-->7-->2-->9



遍历练习题目

编程实现：

输入下图，按深度优先遍历和广度优先遍历输出图的结点。

输入：

8 10

1 4

1 5

1 6

2 6

2 7

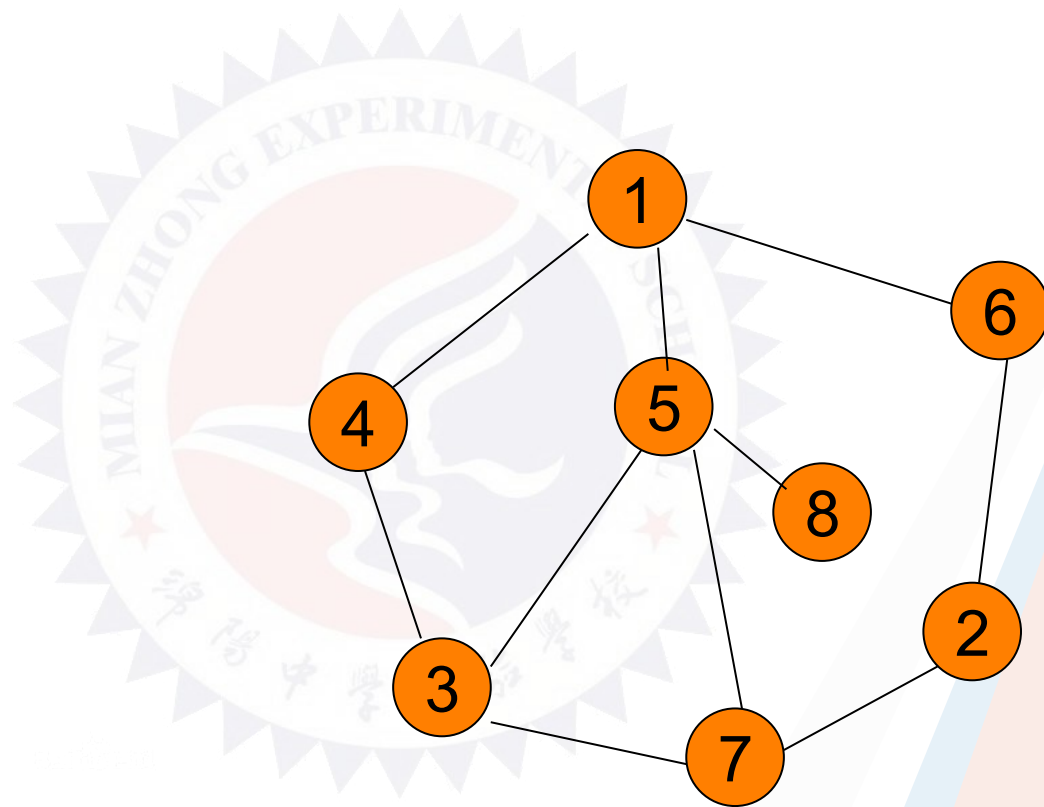
3 5

3 4

3 7

5 7

5 8





→ 犯罪团伙

问题描述：

警察抓到了 n 个罪犯，警察根据经验知道他们属于不同的犯罪团伙，却不能判断有多少个团伙，但通过警察的审讯，知道其中的一些罪犯之间相互认识，已知同一犯罪团伙的成员之间直接或间接认识。有可能一个犯罪团伙只有一个人。请你根据已知罪犯之间的关系，确定犯罪团伙的数量。

已知罪犯的编号从1至 n 。

输入：

第一行： n (≤ 1000 , 罪犯数量) ,

第二行： m (< 5000 , 关系数量)

以下若干行：每行两个数： i 和 j ，中间一个空格隔开，表示罪犯 i 和罪犯 j 相互认识。

输出：

一个整数，犯罪团伙的数量。



犯罪团伙样例

样例输入：

```
11
8
1 2
4 3
5 4
1 3
5 6
7 10
5 10
8 9
```

输出：

3

说明：共三个犯罪团伙。





➔ 犯罪团伙参考代码





→ 哈密顿路

定义：经过图中每个点恰好一次的路

经典的NP完全问题，尚未发现多项式算法...

常见解决方法：

- 状态压缩dp（点数比较少的时候，比如 $N \leq 15$ ）
- 利用问题特殊性（如某些特殊的图存在多项式算法）

NOI2019

→ 哈密顿路

邮递员在送信时，为了节省路途，自己规定：每次总是从 n 个村子中选择其中一个合适的村子出发，途中每个村子仅且经过一次，送完所有的信。已知各个村子的道路连通情况。请你帮邮递员选择一条合适的路线。

输入：

第一行：整数 n ：村子的个数。

接下来是一个 $n*n$ 的0、1矩阵，表示 n 个村子的连通情况，如： $a[l,j]=1$ ，表示第 i 和第 j 个村子之间有路可走，如果 $a[l,j]=0$ ，表示他们之间无路可走。

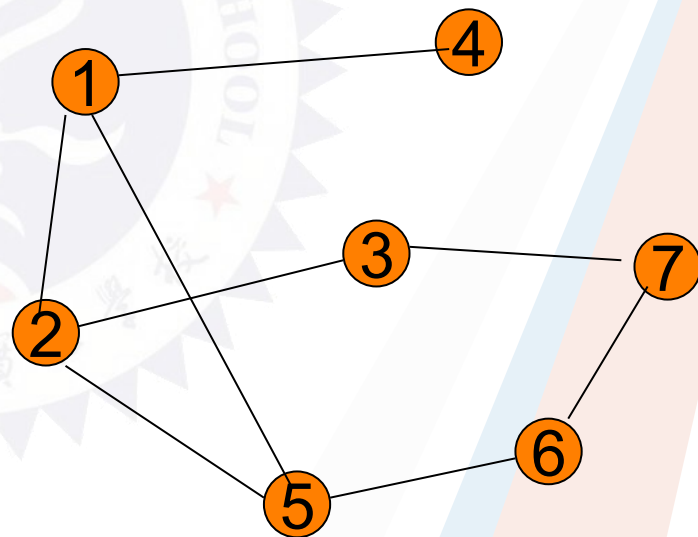
输出：一条可行的路线

输入：

```
7
0 1 0 1 1 0 0
1 0 1 0 1 0 0
0 1 0 0 0 0 1
1 0 0 0 0 0 0
1 1 0 0 0 1 0
0 0 0 0 1 0 1
0 0 1 0 0 1 0
```

输出：

```
2 3 7 6 5 1 4
```



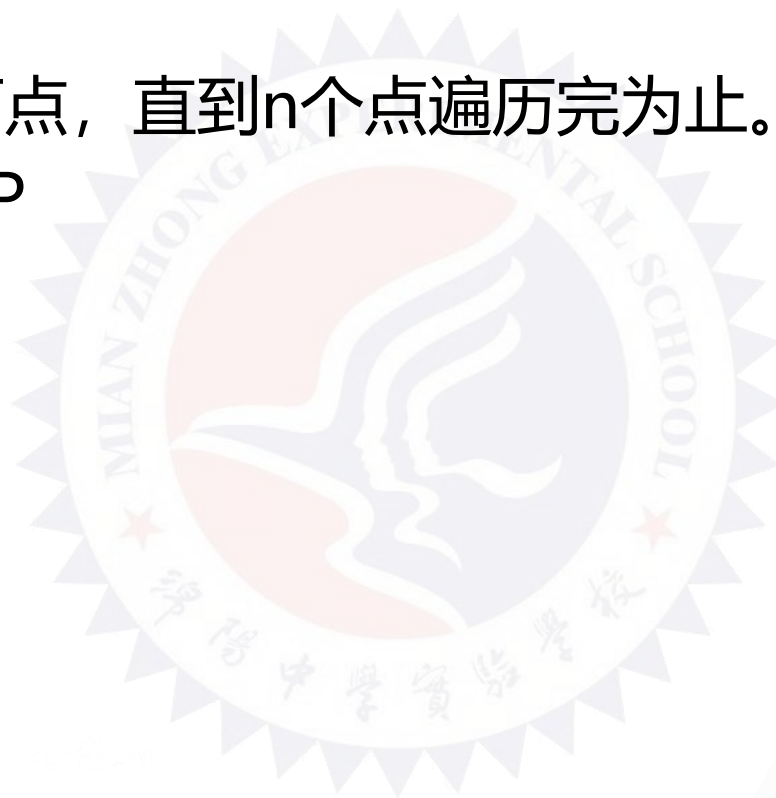


➔ 汉密顿路---DFS

方法一：暴力搜索

- 1、枚举顶点
- 2、然后dfs遍历点，直到n个点遍历完为止。

方法二：状压DP



→ 拓扑排序

拓扑排序是对有向无环图(Directed Acyclic Graph简称DAG)求出一个顶点序列。

对于任意边 $(u,v) \in E$, u 在序列的位置总在 v 之前。

拓扑排序方法如下:

- (1)从有向图中选择一个没有前驱(即入度为0)的顶点并且输出它.
- (2)从图中删去该顶点,并且删去从该顶点发出的全部有向边,并且更新点的入度.
- (3)重复上述两步,直到剩余的图中不再存在没有前趋的顶点为止.

拓扑排序可能不唯一!

用途:

- 1.用于判断有向图是否存在环**
- 2.有向无环图上, 按照拓扑序进行DP**



信息传递 NOIP 2015 day1

问题描述

有 n 个同学（编号为1到 n ）正在玩一个信息传递的游戏。在游戏里每人都有一个固定的信息传递对象，其中，编号为 i 的同学的信息传递对象是编号为 T_i 的同学。游戏开始时，每人都只知道自己生日。之后每一轮中，所有人会同时将自己当前所知的生日信息告诉各自的信息传递对象（注意：可能有人可以从若干人那里获取信息，但是每人只会把信息告诉一个人，即自己的信息传递对象）。当有人从别人口中得知自己的生日时，游戏结束。

请问该游戏一共可以进行几轮？

信息传递

输入格式：输入文件名为，输入共2行。

第1行包含1个正整数 n ，表示 n 个人。

第2行包含 n 个用空格隔开的正整数 $T_1, T_2, \dots, T_n$ ，其中第 T_i 个整数表示编号为 i 的同学的信息传递对象是编号为 T_i 的同学， $T_i \leq n$ 且 $T_i \neq i$ 。数据保证游戏一定会结束。

输出格式：输出文件名为，输出共1行，

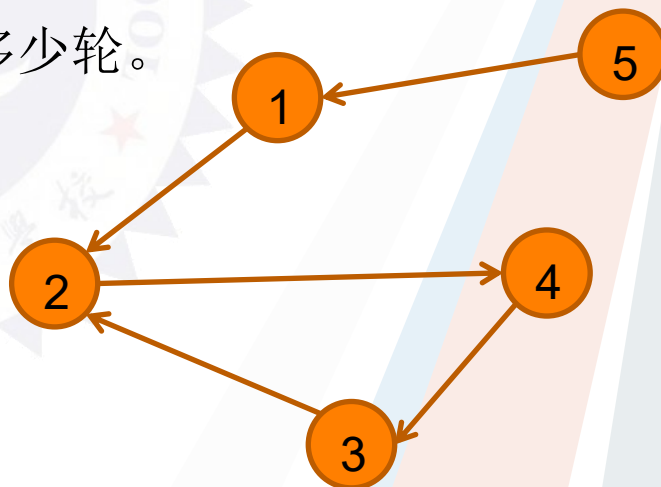
包含1个整数，表示游戏一共可以进行多少轮。

样例输入

5
2 4 2 3 1

样例输出

3



信息传递---题解分析

分析发现：

其实就是求解有向图中所有环中点数最少的环。

如何求解最小环呢？

不妨设每个点 i 有一条边连向点 $t[i]$, 我们可以枚举从点 s 出发, 然后一直走下去, 并且每经过一个点就标记这个点.

如果我们走到了一个已经标记了的点, 此时有两种情况:

1. 这个点在我们从 s 出发走出的这条路径上, 这说明我们找到了一个环, 那么用这个环的大小更新答案即可;
2. 否则, 我们已经处理过这个点了, 直接退出, 枚举下一个起点.

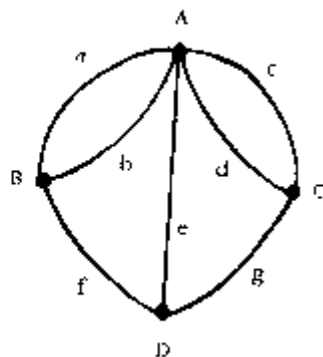
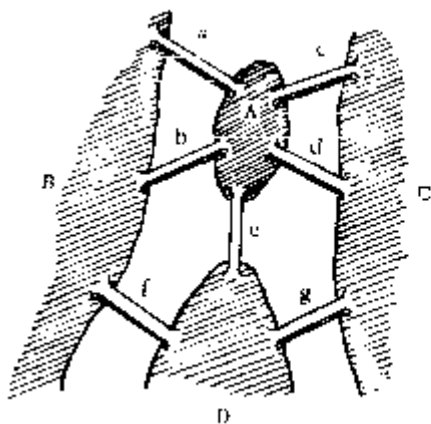
显然, 每个点只会被走一次, 因此时间复杂度是 $O(n)$ 的.



→ 欧拉路

哥尼斯堡七桥问题

背景:18世纪著名古典数学问题之一。在哥尼斯堡的一个公园里,有七座桥将普雷格尔河中两个岛及岛与河岸连接起来(如图)。问是否可能从这四块陆地中任一块出发,恰好通过每座桥一次,再回到起点?欧拉于1736年研究并解决了此问题,他把问题归结为如下右图的“一笔画”问题,证明上述走法是不可能的。





欧拉路

- ✧ 欧拉路: 给定无孤立结点图 G , 若存在一条路, 经过图中每边一次且仅一次, 该条路称为欧拉路。如果起点和终点相同, 叫做欧拉回路。
- ✧ 无向图存在欧拉路的条件
 - 如果图连通, 且所有点度数都是偶数, 则有欧拉回路。
 - 如果图连通, 且恰有2个点度数是奇数, 则有欧拉路。且欧拉路的起始和终止点是这2个点。
- ✧ 有向图存在欧拉路的条件
 - 如果图连通, 且每个点的入度等于出度, 则有欧拉回路
 - 如果图连通, 且恰有一个点出度比入度大1, 另一个点的出度比入度小1, 则存在欧拉路, 并且这2个点分别为起点终点。



→ 欧拉路

- ✧ 简单的证明一下无向图欧拉回路判定条件的正确性，其他几种和这个差不多，自行脑补。
- ✧ 必要性：设 G 具有欧拉回路 $v_0e_1v_1e_2v_2...e_iv_ie_{i+1}...e_kv_0$ 。对于中间点，每个点关联两条边，该点度数每次 $+2$ ；起点在开始和最后关联一条边，起点度数还是 $+2$ ，所有点度数为偶数。因为欧拉路经过图上每个点，所以图连通。
- ✧ 充分性：(1)选择一个起点dfs最终回到起点，得到一个回路 L_1 。如果 L_1 经过图中每条边，那么 L_1 就是欧拉回路。(2)否则从图 G 中去掉 L_1 ，得到子图 G' ， G' 每个点的度数为偶数。因为原图连通，所以 L_1 与 G' 至少一个有一个点 v_i 重合，在 G' 中选择 v_i 做起点dfs，得到回路 L_2 ，如果 L_1 和 L_2 套在一起得到图 G ，那么得到欧拉回路。否则继续进行(2)操作，得回路 L_3 ，依次类推。



→ 套圈法

- ✧ 对于欧拉图，从一个节点出发，随便往下走（走过之后需要标记一下，下次就不要来了），必然也在这个节点终止（因为除了起始节点，其他节点的度数都是偶数，只要能进去就能出来）。这样就构成了一个圈，但因为是随便走的，所以可能会有些边还没走过就回来了。我们就从终止节点逆着往前查找，直到找到第一个分叉路口，然后从这个节点出发继续上面的步骤，肯定也是可以找到一条回到这个点的路径的，这时我们把两个圈连在一起。当你把所有的圈都找出来后，整个欧拉回路的寻找就完成了。
- ✧ 算法思想：从起点开始，任意找路，最后肯定是停在终点而无法继续，得到一条路径： $P[1], P[2], P[3], \dots, P[n]$ ，然后将图中剩下的以 $P[1]$ 为起点和终点的圈，以 $P[2]$ 为起点和终点的圈，……，以 $P[n]$ 为起点和终点的圈，插入路径中。假设以 $P[i]$ 为起点和终点的圈为 $Q[1], Q[2], Q[3], \dots, Q[m]$ ，则插入路径中得到： $P[1], P[2], P[3], \dots, P[i](Q[1]), Q[2], Q[3], \dots, Q[m], P[i+1], \dots, P[n]$ 。

→ 套圈法

实现：DFS一遍，把沿途的边倒着输出就行了。

DFS搜索 思想求解欧拉回路的思路为：利用欧拉定理判断出一个图存在欧拉通路或欧拉回路后，选择一个正确的起始顶点，用**DFS**算法遍历所有的边（每条边只遍历一次），遇到走不通就回退。在搜索前进方向上将遍历过的边按顺序记录下来。这组边的排列就组成了一条欧拉通路或回路。

```
cnt=0;
dfs(int u){
    for(每个e.from!=u&&e未被访问) {
        标记e被访问;
        dfs(e.to);
        ans[cnt++] = e;
    }
}
```

倒序输出ans中的每条边，即为欧拉路。

这个算法适应性很强，有向图无向图回路通路都适用哟~

The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two data series is visible on the right side.

Thank you!